



Pure-Gemma Evolution Matrix

Agentic Workflows for Python Test Generation

Politecnico di Torino

Johnprice Osagie · Matteo Gastaldello · Erestina Vreto

Large Language Models for Software Engineering (2025 / 2026)

Architecture

Orchestration

LangGraph: Stateful multi-agent workflows and complex logic routing.

LLM Backend

Google Vertex AI / AI Studio: Testing across Gemma family (1B to 31B parameters).

Execution & Data

Execution

Custom Instrumentation: AST parsing for fine-grained tracking.

Multi-process isolation with hardware kill-switches (for API rate limits).

Data Engineering

Results Pipeline: 265-row matrix using pandas across 8 models, 11 architectures, and 4 strategies.



The "Hardest" Logic Subset

Targeting Pure Reasoning

Isolated a 25-task subset from HumanEval consisting of purely algorithmic challenges (no I/O, no network calls).

The Final Range

Selected tasks 139 to 163, representing the most mathematically and logically complex problems in the dataset.

Why EvalPlus (Breaking Memorization)

Beyond Standard Tests

Vanilla HumanEval only has ~7 simple tests. LLMs often "memorize" these standard solutions. EvalPlus is an evolution of HumanEval with up to 700+ tests per task.

Massive Amplification

EvalPlus augments this with ~80× more test cases per task.

HumanEval/140 — `fix_spaces(text)`

Goal: Replace single spaces with '_' and groups of >2 spaces with '- '.

Examples:

"Example 1" -> "Example_1"

" Example 3" -> "_Example-3"

HumanEval/143 — `words_in_sentence(s)`

Goal: Return only words whose length is a prime number.

Examples:

"This is a test" -> "is"

"lets go for swimming" -> "go for"

Experimental Setup

Strategies

- **Zero-shot:** Plain task description, no example
- **Few-shot:** 2 solved HumanEval examples as priming
- **CoT:** Chain-of-Thought — "Let's think step by step"
- **SCoT:** Structured CoT — reasoning → plan → code

Implementation & Goals

- Routed via `apply_reasoning_style()` in `prompts.py`
- **Goal:** Isolate prompting impact vs. agent architecture

Key Findings

- **4B:** Few-shot dominates performance
- **27B / 4-31B:** SCoT provides significant boost
- **12B:** Hurt by CoT — verbose but inaccurate

Observations & Constraints

Parameters & Output

- **4B:** Cleanest code blocks, lowest parser error
- **27B+:** Prose preamble requiring regex stripping
- **Temp:** Deterministic vs Probabilistic/"Creative"

Experimental Control

- Standardized system prompt for multi-agent fairness
- **Ablation:** Prompt change alone → +6% PR delta

Identified Risks

- **Few-shot:** Risk of example leakage/over-fitting
- **CoT:** Risk of hallucinated reasoning justifications
- Prompting ≠ replacement for agent feedback loops



Baseline

Single LLM call without iterative loops. Used as the Control Condition.

Generation

Specialized patterns for code synthesis.

Agents: Swarm, Atomic Swarm, CoA, SOA.

Validation

Quality gates and error correction.

Agents: Consensus, Self-Healing, Actor-Critic.

Adversarial

Competitive debugging and edge-case discovery.

Agents: Adversarial, Competitive.

Hybrid

Integrated pipelines combining generative reasoning with automated validation.

System Framework

- Common state managed by **LangGraph StateGraph**
- Conditional edges transition on test **pass** / **fail**
- Max iteration cap (5–10 rounds) prevents infinite loops
- **Matrix:** 5 models × 11 agents × 4 prompts = **220 configs**

Pseudocode — main feature

```
# baseline.py – control condition
def run(task, model):
    return model.generate(task.prompt)
```



Swarm

N parallel generators with voting.

Aggregation: Vote, test-filtering, or LCS.

Optimal N: N=10 for small model variance.

Atomic Swarm

Workers + unit tests.

Token Premium: Swarm cost scales $O(N)$.

CoA

Sequential (Planner → Coder → Tester → Fixer).

Error Compounding: Sequential hallucination risk.

SoA

Multi-turn collab (Architect, Coder, Reviewer).

Pseudocode — Main Implementation Patterns

```
# Swarm
def swarm(task, model, N=10):
    cands = [model.generate(task.prompt)
    for _ in range(N)]
    final = aggregate(cands,
    strategy="dedupe+compose")
    return final
```

```
# Atomic Swarm
def atomic_swarm(task, model):
    base = model.generate(task.prompt +
    " // typical case")
    edge = model.generate(task.prompt +
    " // edge case")
    type_c = model.generate(task.prompt +
    " // type/invalid")
    return merge([base, edge, type_c])
```

```
# CoA (Chain of Analysis)
def coa(task, mgr, worker):
    dims = mgr.analyze(task.prompt)
    suite = ""
    for d in dims:
        suite += worker.generate(
        task.prompt + f" // focus: {d}")
    return suite
```

```
# SOA (Service-Oriented)
def soa(task, orch, spec):
    specs = orch.identify(task.prompt)
    tests = []
    for s in specs:
        tests.append(spec.generate(
        task.prompt + f" // exp: {s}"))
    return combine(tests)
```



Consensus

- 3 proposers + 1 judge model
- Judge ranks candidates on correctness + clarity
- Highest performance ceiling

Self-Healing

- Cycle: generate → test → feedback → regenerate
- Max 5 rounds; early abort on no-change
- Most token-efficient strategy

Actor-Critic

- Dynamic duo: Tester defends, Hacker breaks
- Hacker targets candidate failure points
- Excellent for surfacing edge-case blind spots

Pseudocode — Main Implementation Patterns

```
# Consensus
def consensus(task, proposers, judge):
    cand = [p.generate(task.prompt)
    for p in proposers]
    return judge.select(cand,
    criteria="correctness")
```

```
# Self-Healing
def self_heal(task, model, rounds=5):
    code = model.generate(task.prompt)
    for _ in range(rounds):
        if passes(code): return code
        errs = run(code)
        code = model.generate(p, errors=errs)
    return code
```

```
# Actor-Critic
def actor_critic(task, actor, critic):
    code = actor.generate(task.prompt)
    for _ in range(3):
        fb = critic.review(code)
        if fb == "PASSED": return code
    code = actor.generate(p, feedback=fb)
    return code
```

Adversarial

- Tester defends, Hacker breaks
- Hacker proposes inputs that fail the candidate
- Useful for surfacing edge-case blind spots

Competitive

- Two generators judged head-to-head
- **Strength:** explicit quality signal at every step
- **Weakness:** errors compound; Critic must outrank Actor

Hybrid

- Swarm population followed by Self-Healing
- Aggregation: majority vote / test-based / LCS
- Token cost: linear in N (dominant expense)
- Risk: correlated hallucinations across workers

Pseudocode — Main Implementation Patterns

```
# Adversarial
def adversarial(task, hacker, tester):
    mutated = hacker.mutate(task.source)
    tests = tester.generate(task.prompt_for(mutated))
    res = run_tests(mutated, tests)
    caught = (res.failed > 0 or res.errors > 0)
    return {"mutated": mutated, "caught": caught}
```

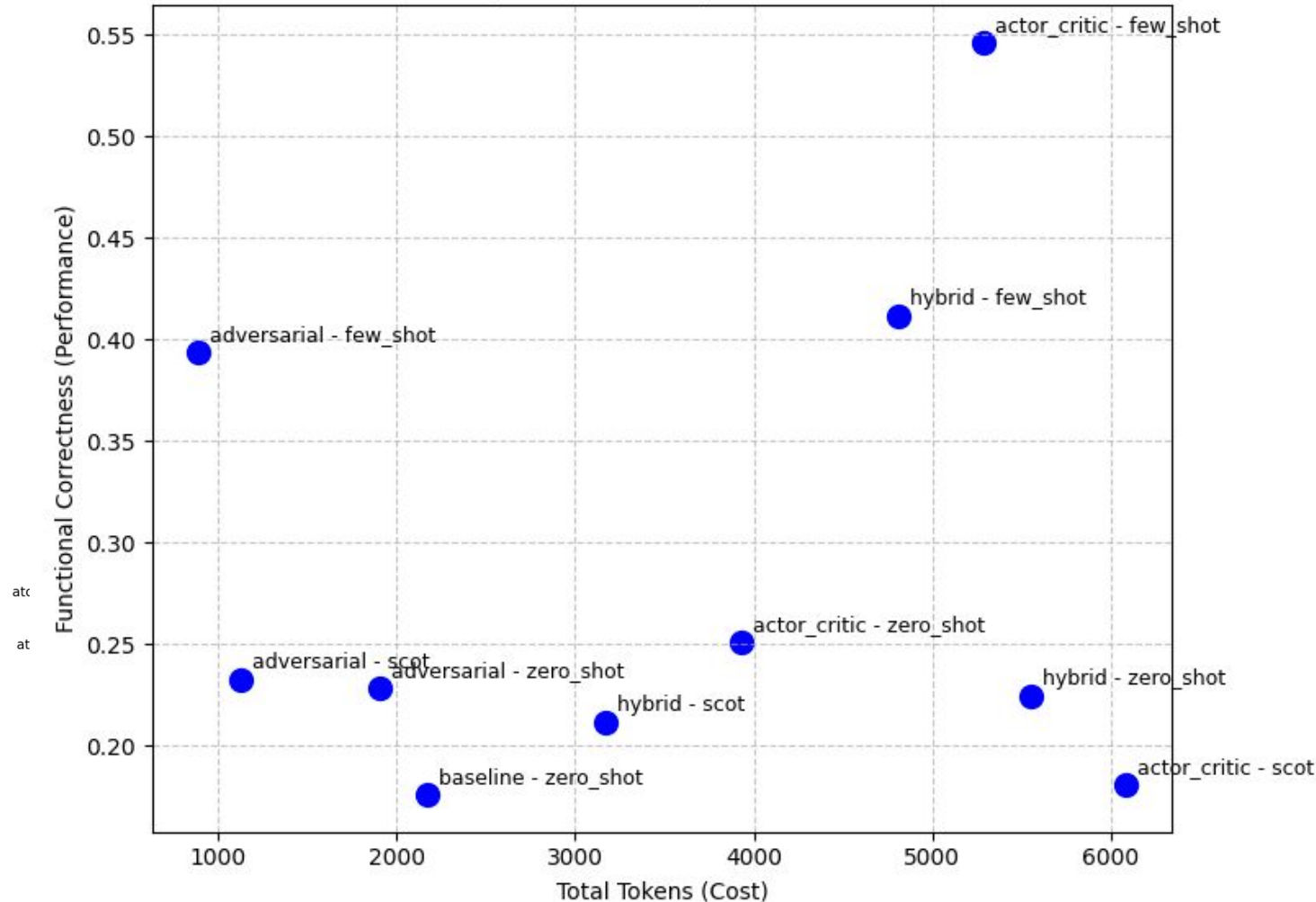
```
# Competitive
def competitive(task, models):
    gens = {m: m.generate(task.prompt) for m in models}
    scores = {}
    for m, code in gens.items():
        m_res = run_tests(task.source, code)
        scores[m] = m_res.functional_correctness * 100
    return {"winner": argmax(scores)}
```

```
def hybrid(task, gen_model, evolve_model):
    # Diverse variations (swarm)
    pop = [gen_model.generate(task.prompt + f" // v{i}") for i in range(2)]

    # Evolve into final solution
    best = evolve_model.generate("Merge:\n" +
    "\n---\n".join(pop))

    return best
```

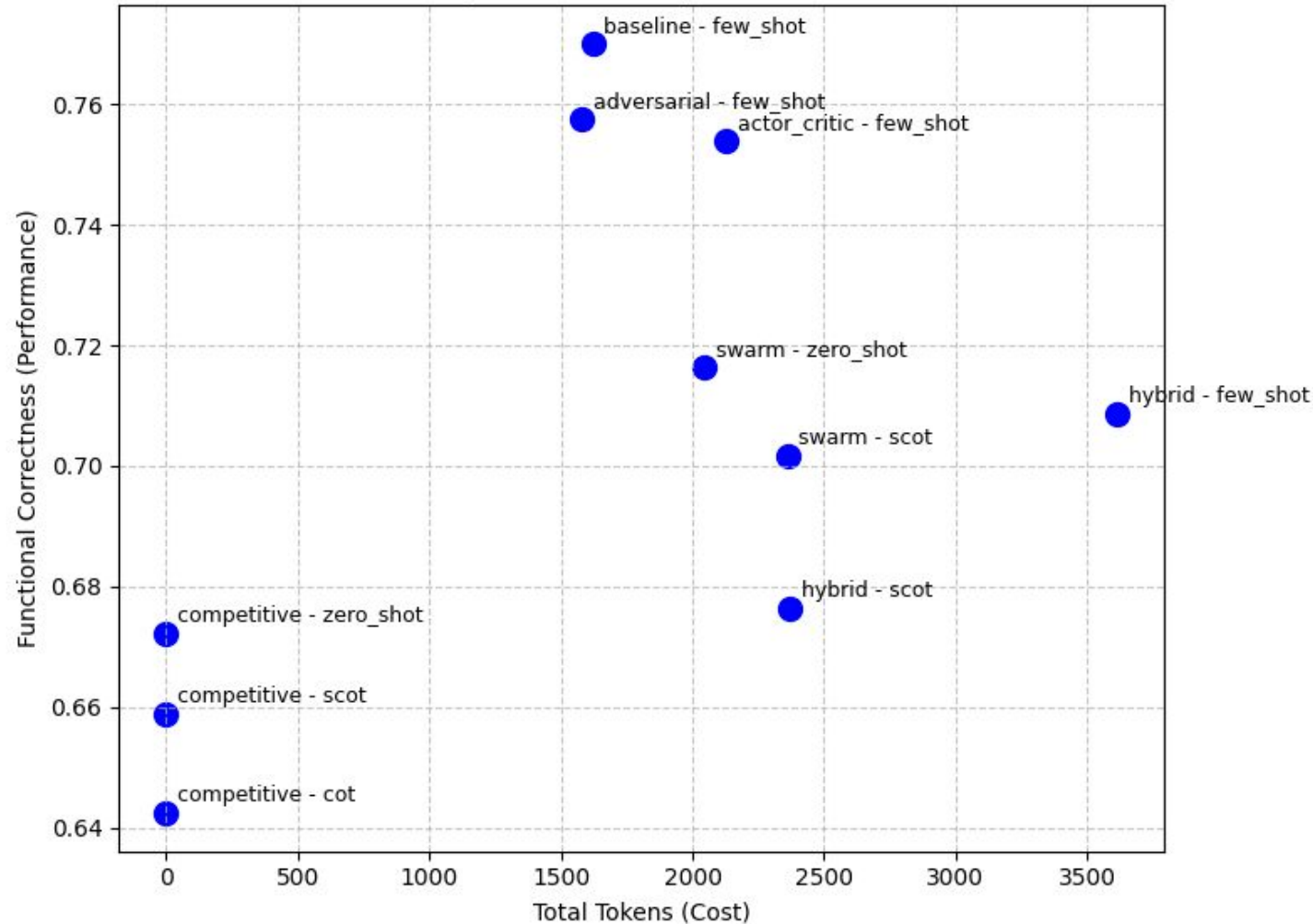

Efficiency Trade-offs (Top 10) - GEMMA-1B



Considerations:

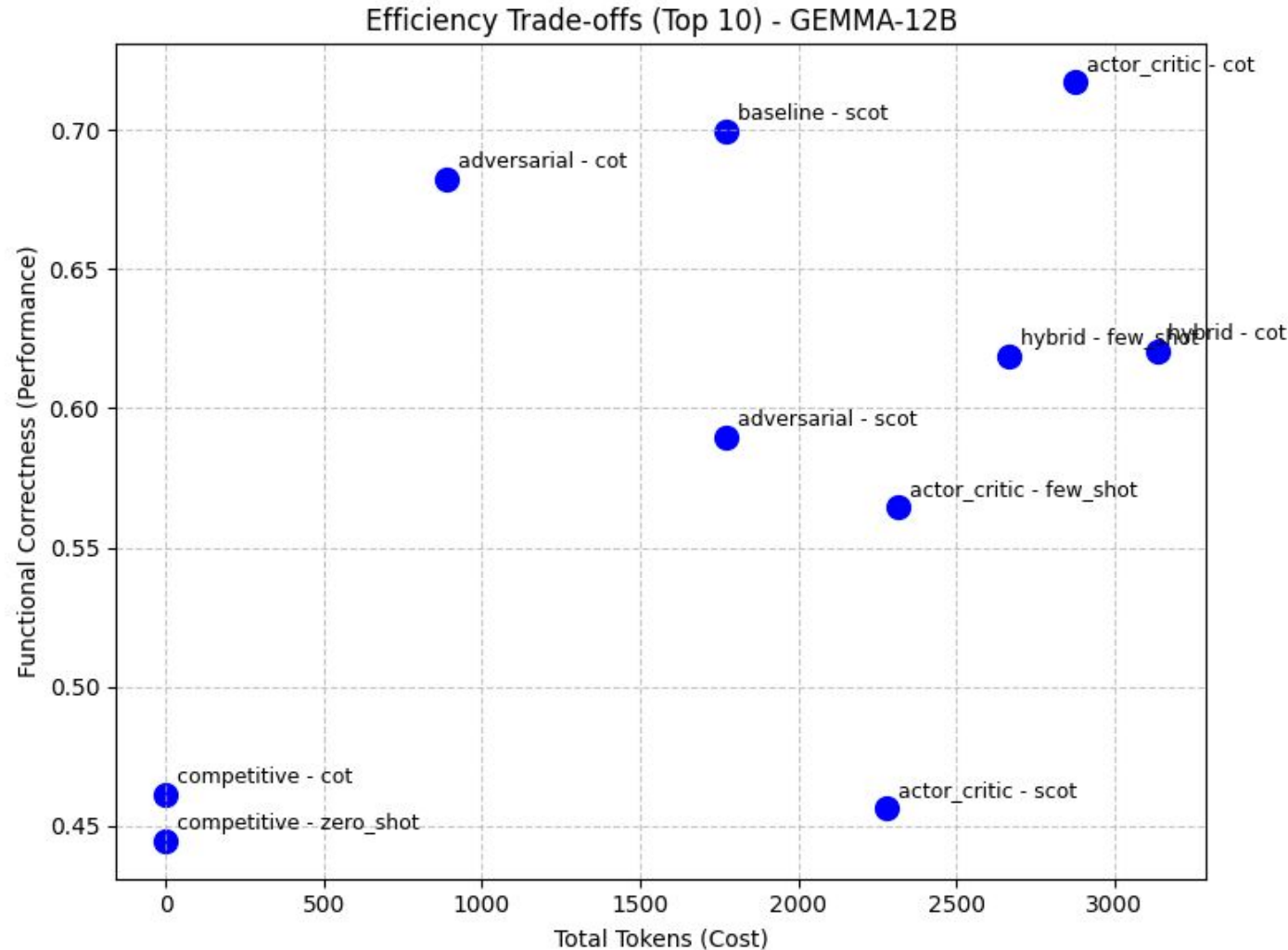
- **Top Performer: actor_critic + few_shot**
Achieves **54.6%** Pass Rate (3x increase over the **zero_shot** baseline).
- **Few-Shot is essential:** because of the lack of abstraction, providing concrete examples is critical for structuring code correctly.
- **Architectural Limits** Overly fragmented multi-agent workflows (e.g., **atomic_swarm**, **coa**) overwhelm the model, collapsing to 0% accuracy.
- **Token Consumption Trade-off** Consumes ~2.5x more tokens (~5300 vs ~2160), but completely triples the success rate.

Efficiency Trade-offs (Top 10) - GEMMA-4B



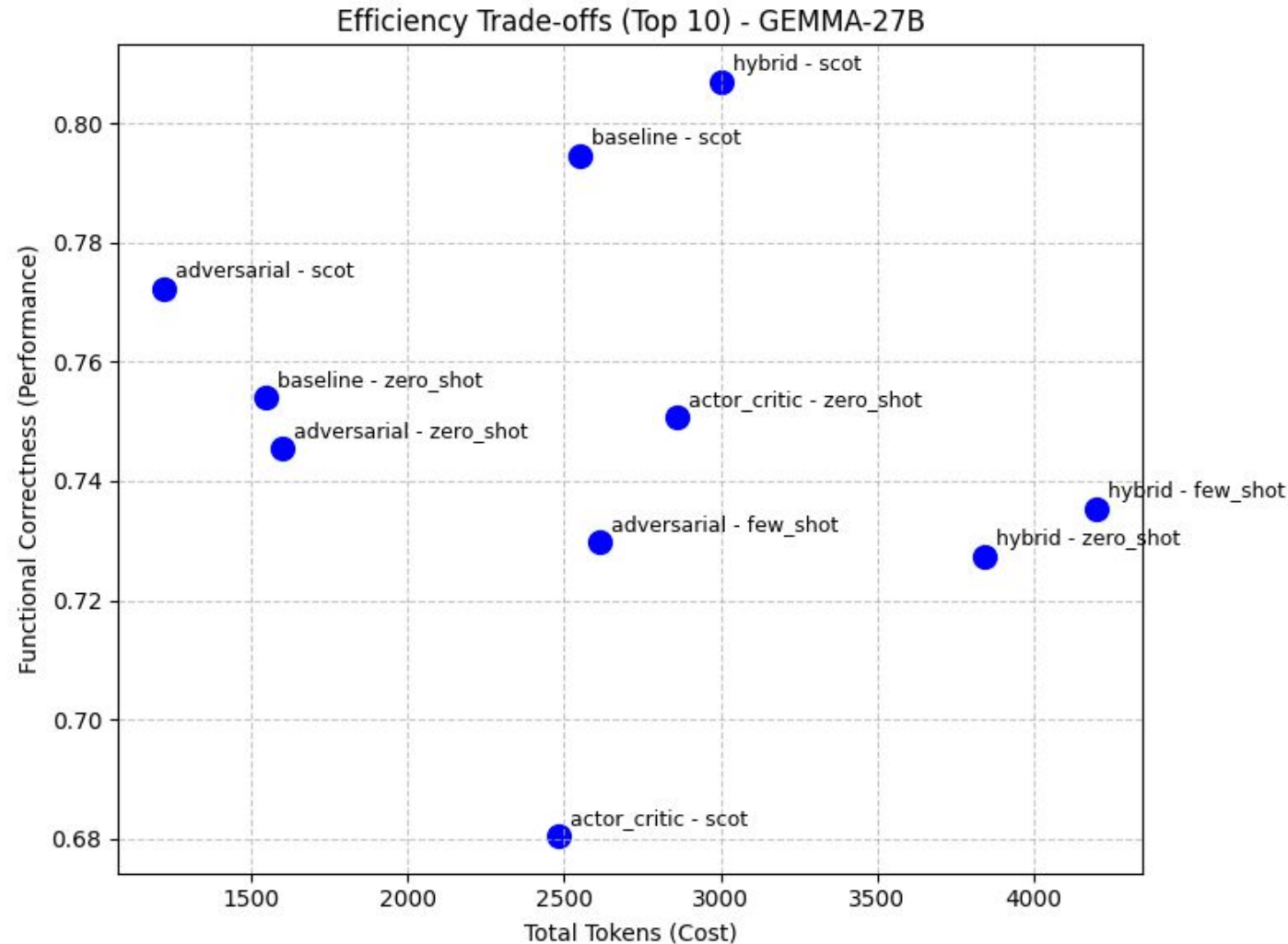
Considerations:

- **Simplicity Wins:** **baseline + few_shot** Peak accuracy at **77.0%** (+15% over **zero_shot**).
- **actor_critic** and **adversarial** approaches are competitive (~75%) but introduce unjustifiable system overhead.
- **Chain of Abstraction** (CoA) workflows fail entirely (~1.3%). Forcing a 4B model to write and translate pseudo-code destroys context.
- Providing examples actually *reduces* token usage (~1620 vs ~2527). The model avoids hallucinations and writes concise code.



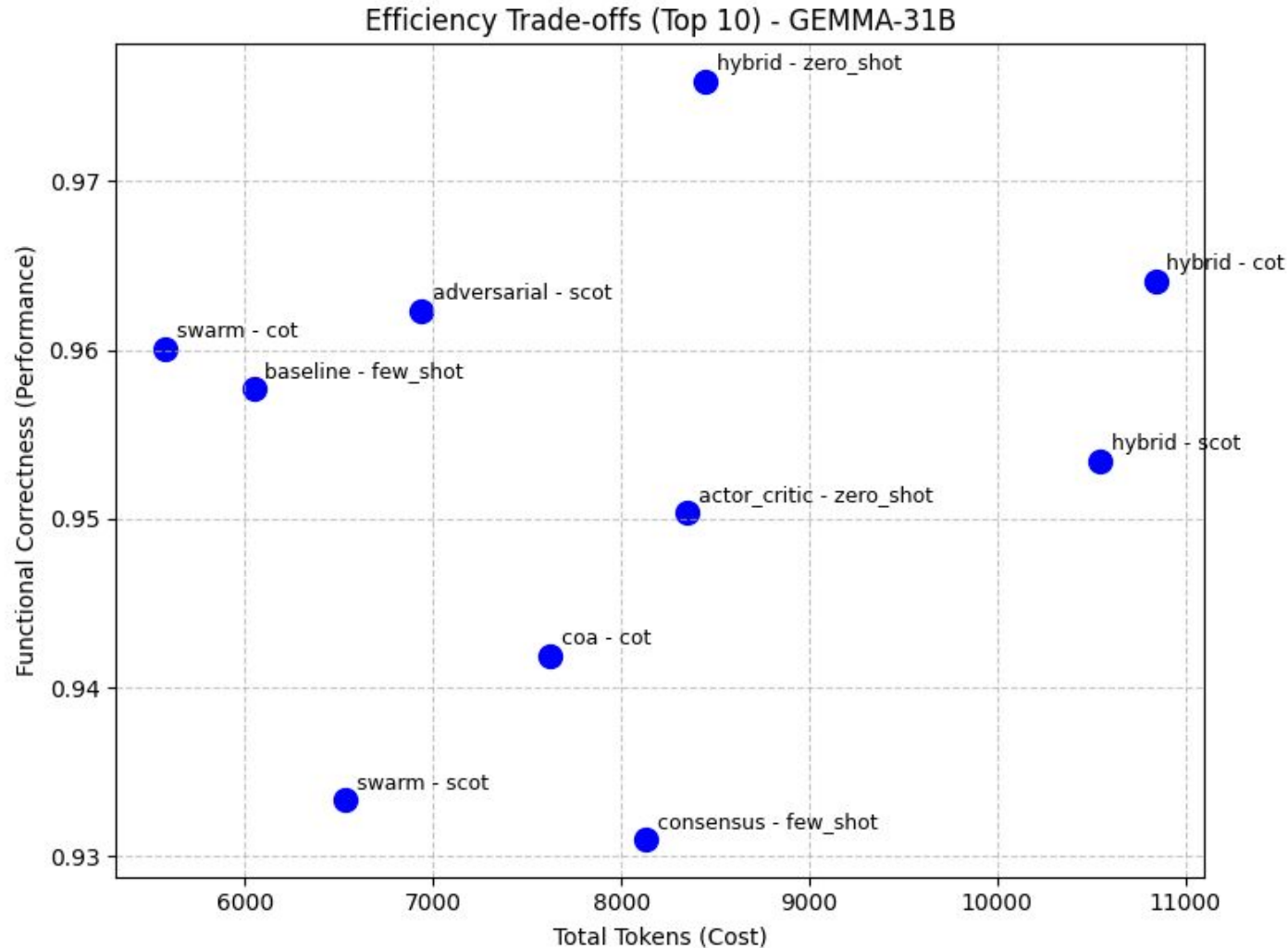
Considerations:

- **Size Isn't Everything:** `zero_shot` baseline severely underperforms at just **24.6%**.
- **CoT:** `actor_critic` + `cot` Forcing the model to "think aloud" (Chain of Thought) jumps accuracy to **71.7%** (nearly a 3x jump).
- **Heavy Abstraction Fails** Complex multi-agent architectures (`swarm`) or pseudo-code translation (`coa`) overwhelm the 12B model, collapsing to ~0-1.6% accuracy.
- **Efficiency:** `adversarial` + `cot` Achieves a **68.2%** accuracy while using only **~891 tokens**—costing almost exactly the same as the weak baseline.



Considerations:

- Massive out-of-the-box performance (`zero_shot` baseline is **75.4%**). Heavy multi-agent systems add marginal gain while doubling token costs.
- **Self-Correction:** The top 3 overall strategies rely entirely on SCoT. Model is able to review and fix its own code.
- `adversarial:scot` achieves a 77.2% accuracy while consuming *fewer* tokens than the raw baseline (~1224 vs ~1547).
- Forcing the model into a `competitive` multi-agent environment crashes accuracy to ~3.4%. The model gets lost trying to follow the rules of the debate.



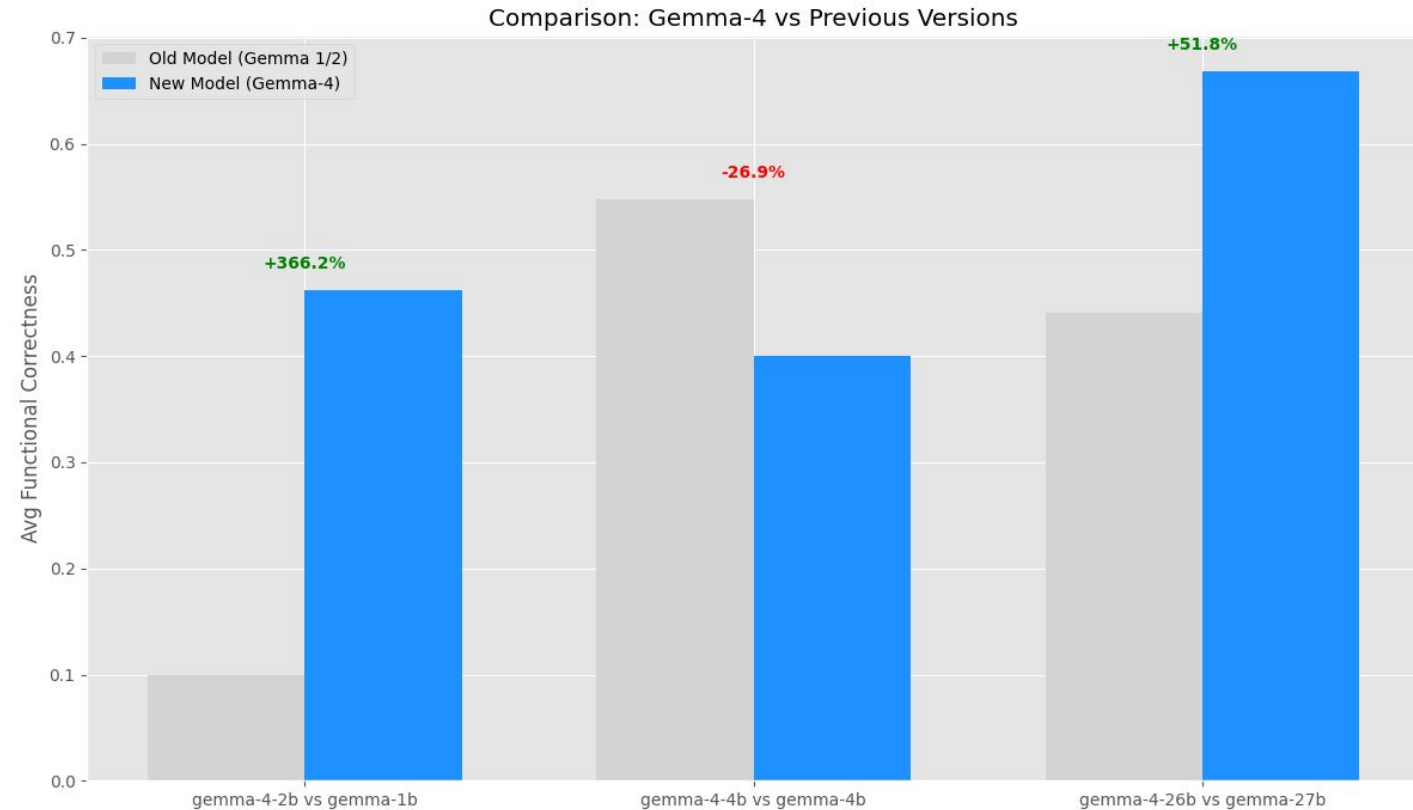
Considerations:

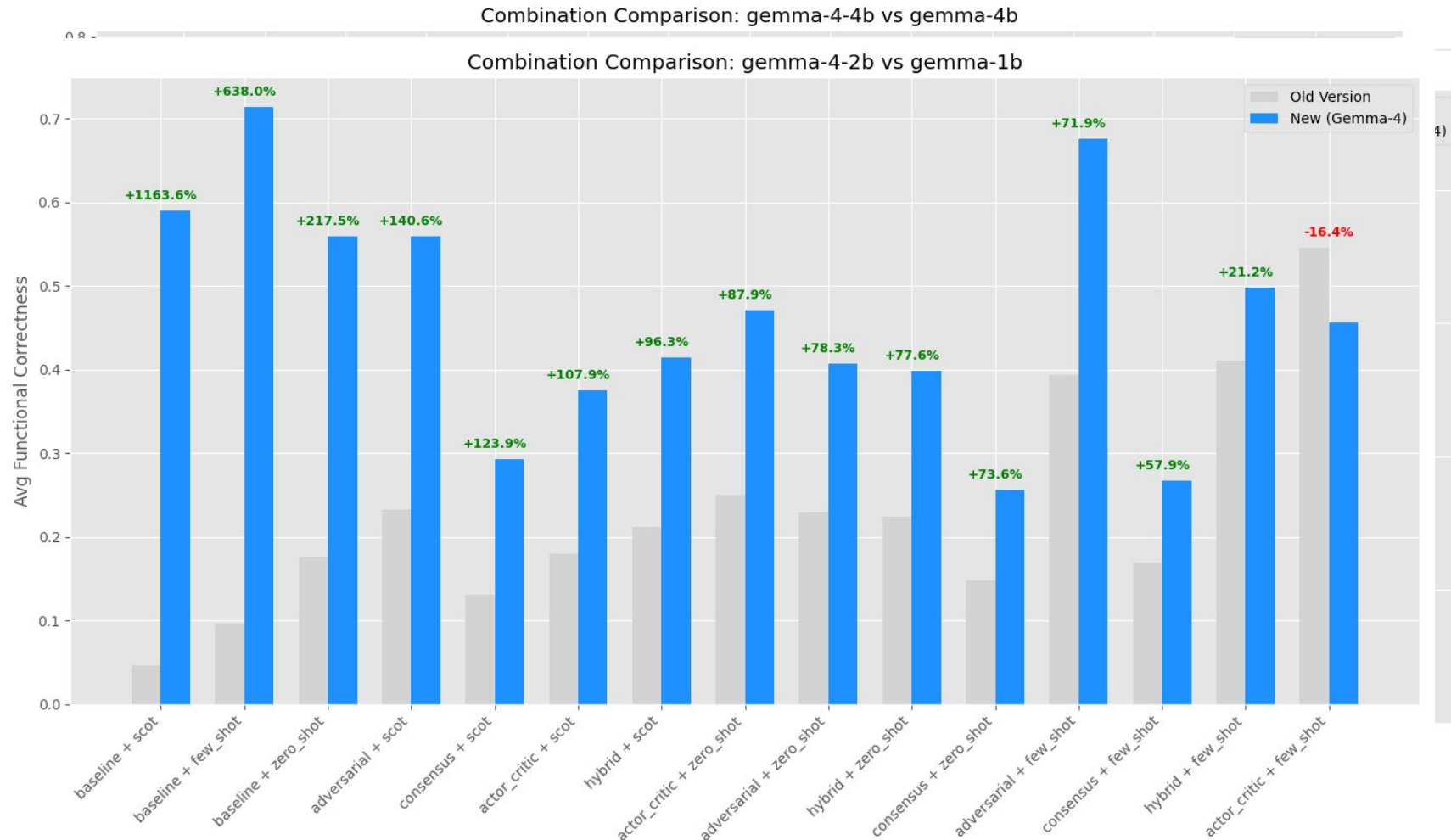
- The unassisted `zero_shot` baseline starts at a massive **88.6%**.
- The `hybrid:zero_shot` architecture reaches a near-perfect **97.6%** accuracy.
- **Few-Shot Penalty:** Providing examples harms performance (e.g., forcing a `few_shot` on the hybrid agent drops accuracy to 54%).
- **Extreme Token Efficiency** Adding agents does not equate to higher cloud costs. `swarm:cot` achieves 96.0% accuracy while consuming *fewer* tokens than the raw baseline (~5575 vs ~6894).
- **Self-repair patterns** destroy the model's native logic, dropping accuracy to ~20-28%.

In April 2026, the Gemma 4 family was introduced.

We tried the best combinations we found in the previous Gen to compare how the new gen would perform:

- The Gemma-4 architecture provides extreme performance boosts for the smallest models and the larger models.
- However, the Gemma-4-4B model underperforms compared to its predecessor





Baselines Beating Agents (Small Tier): The raw, zero-shot capabilities of the new 2B model easily surpass the absolute highest, most heavily agent-assisted peak of the older 1B version.

The Efficiency Trap (Mid Tier): To compensate for its unexpected regression, the new 4B model is forced to rely on token-expensive multi-agent pipelines just to catch up to the old 4B's standard baseline.

The Obsolescence of Agents (Large Tier): The new 26B model is so inherently capable out-of-the-box (~89% zero-shot accuracy) that it renders complex, expensive multi-agent architectures completely unnecessary for these tasks.

Key Insights: Comparative Analysis of Generational Leap

Breakthrough

2B vs. 1B

- Massive performance jump (+366%).
- Expected since we double the size

Disappointment

4B vs. 4-4B

- Global performance drop (-27%).
- Extreme prompt fragility: High-density models suffer "structural collapse".

Efficiency Gap

26B vs. 27B

- Superior raw performance but extremely inefficient.
- Gen 4 consumes ~11x more tokens due to reasoning overhead.

Final Verdict

- Gen 4 models are overall smarter but more "capricious" (more sensitive to architecture) and expensive.

PassRate Optimization

The smallest models show the biggest margin for improvement.

Token Efficiency Leaders

Adversarial and SOA, despite not being the strongest in terms of Pass Rate/Tokens.

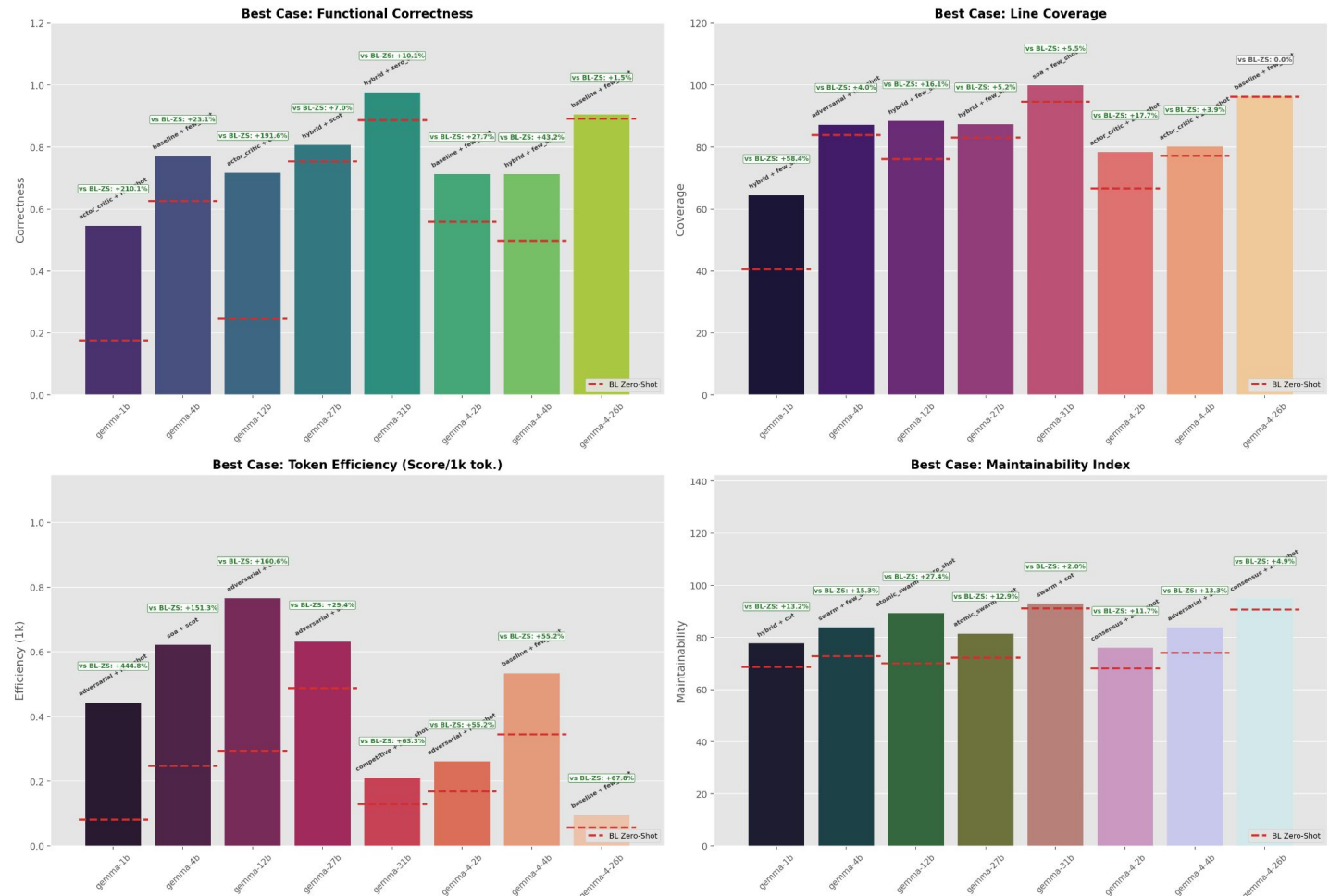
Line Coverage Maxima

Hybrid architectures reach peak levels in many models, while the max is achieved by SOA+few_shot in Gemma 31B.

Maintainability Gains

Complex agents like Consensus or Atomic Swarm show the biggest improvement for MI, leading to modular code.

Best-Case Performance per Model vs Baseline Zero-Shot



Efficiency Collapse

Using complex agents (Consensus/Swarm) in Zero-Shot mode results in an efficiency drop of >85%.

PassRate Degradation

Small models (1B) in high-overhead architectures without Few-Shot support can reduce PassRate by 65%.

Maintainability Pitfalls

Swarm + COT reduces Maintainability Index by up to 37% due to over-engineered logic.

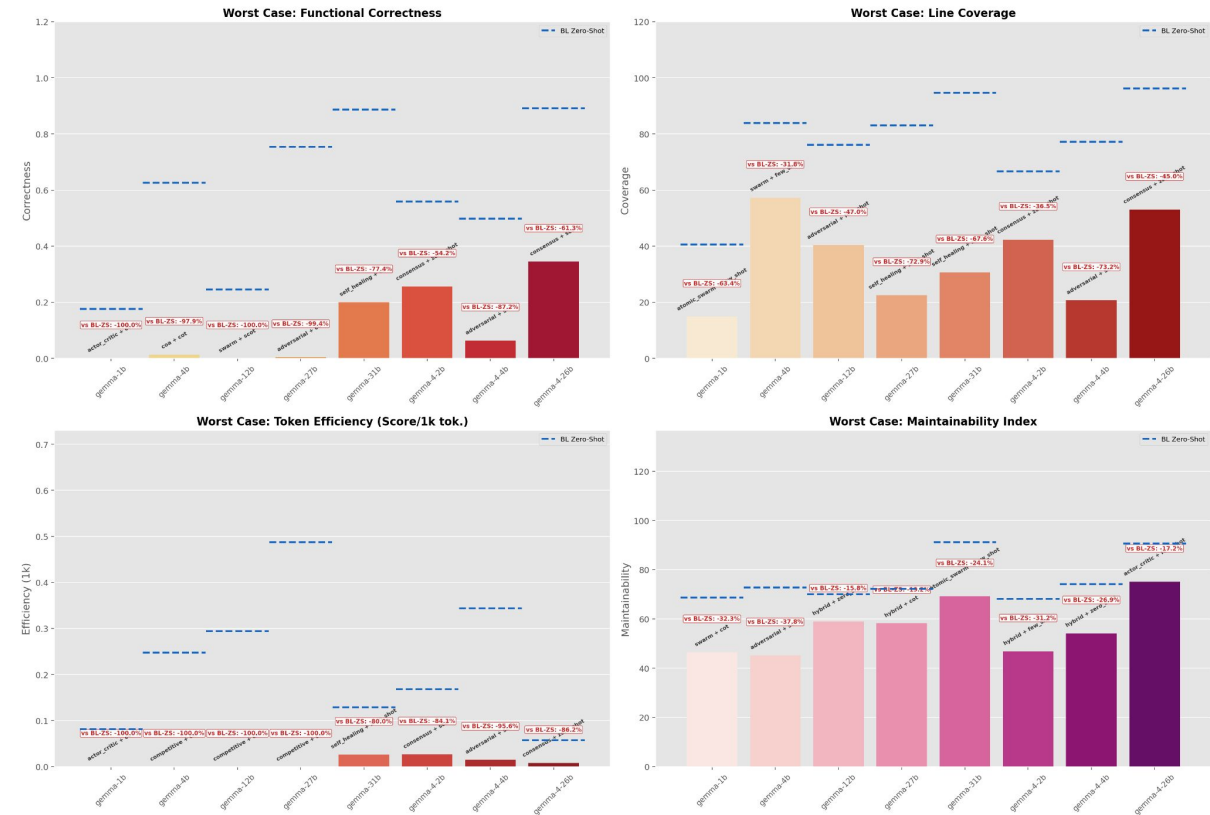
Architecture Overkill

A "Complexity Penalty" occurs where multi-call overhead becomes a liability without sufficient model capacity.

Decoupled Quality

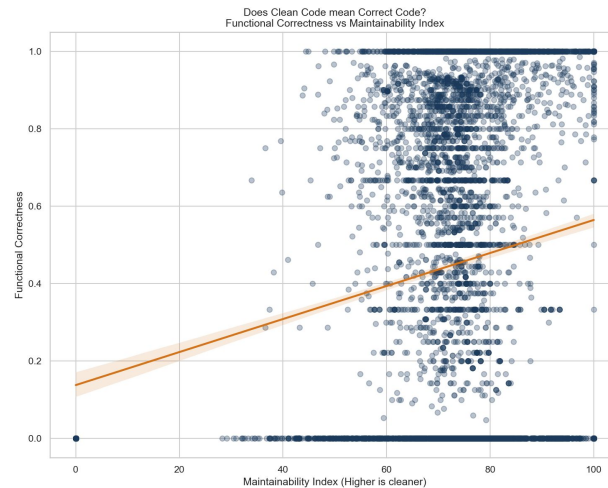
MI evaluates code structure rather than execution, making it less sensitive to logic errors.

Worst-Case Performance per Model vs Baseline Zero-Shot
(each subplot picks the worst agent+style for THAT specific metric)



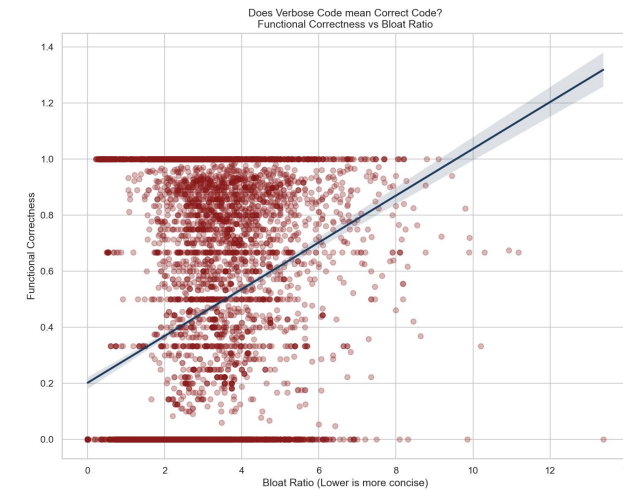
Definition and Formula

- **Definition:** Fraction of EvalPlus tests passed per task.
- **Formula:** $PR = \text{passed_tests} / \text{total_tests}$
- **Advantage:** Catches edge cases that memorized solutions miss.



Interpretation and Thresholds

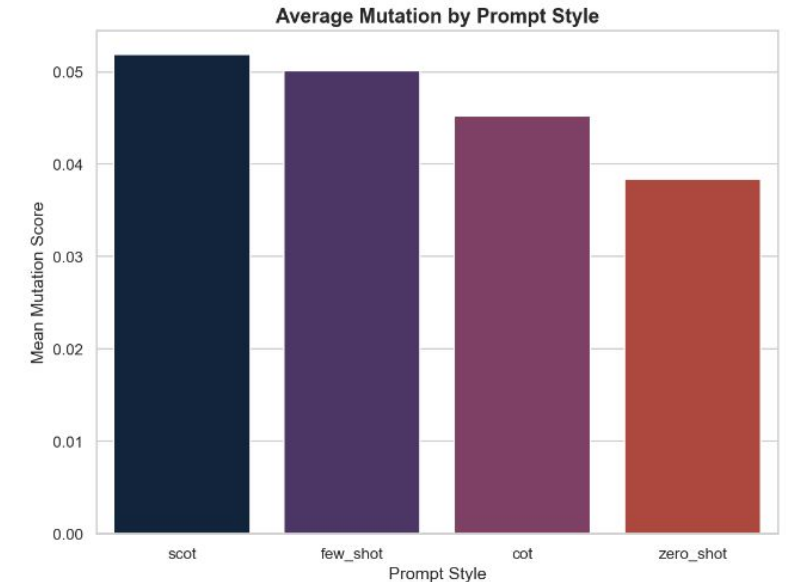
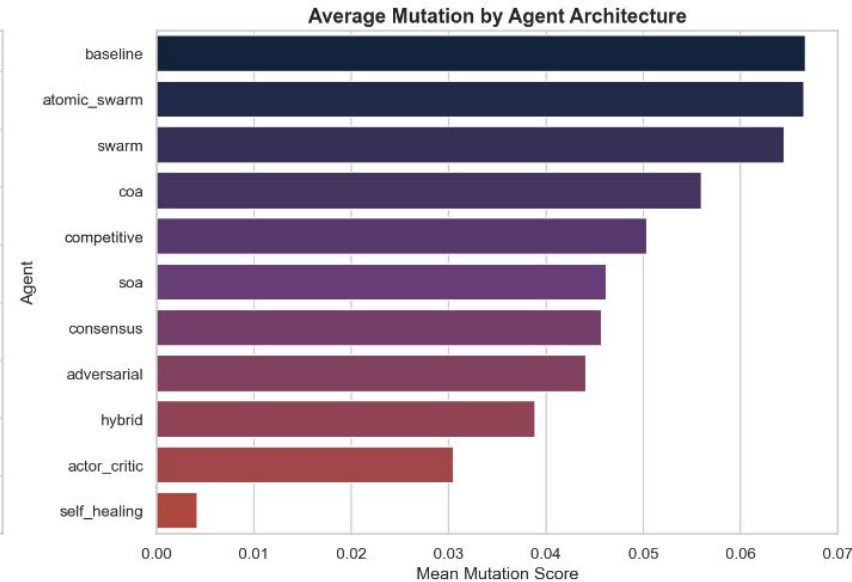
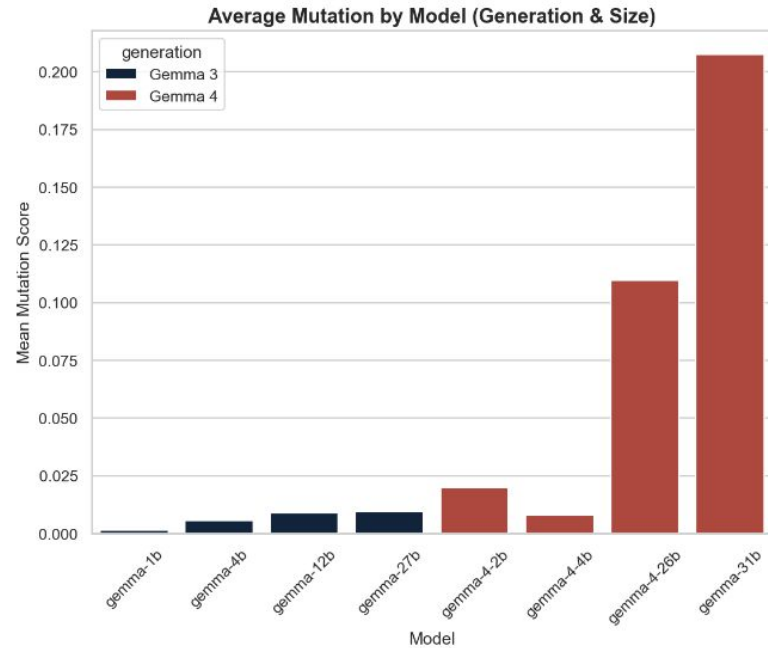
- **> 0.80:** "Production-viable" threshold.
- **< 0.40:** "Unreliable, reject".
- **Primary Metric** for this study



Observation: While low bloat (concise code) is usually preferred, here **PassRate** correlates with high bloat.

Limitations: Measures test behavior, not code quality.

Mutation Score Comprehensive Analysis (Combined View)



Observation: This is the metric where the generational gap is shown the most.

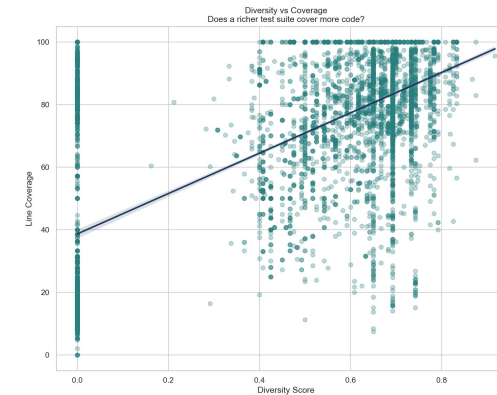
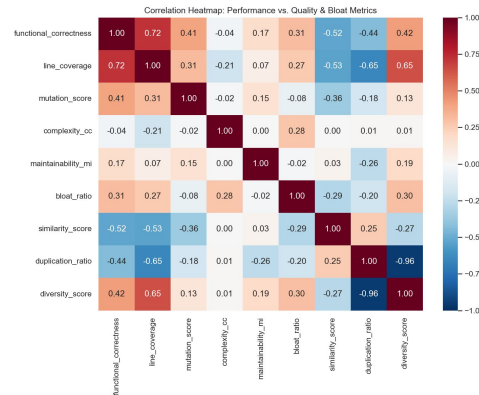
Take-away: Baseline performs best, zero-shot and actor-critic underperforms.

Metric Definitions & Methodology

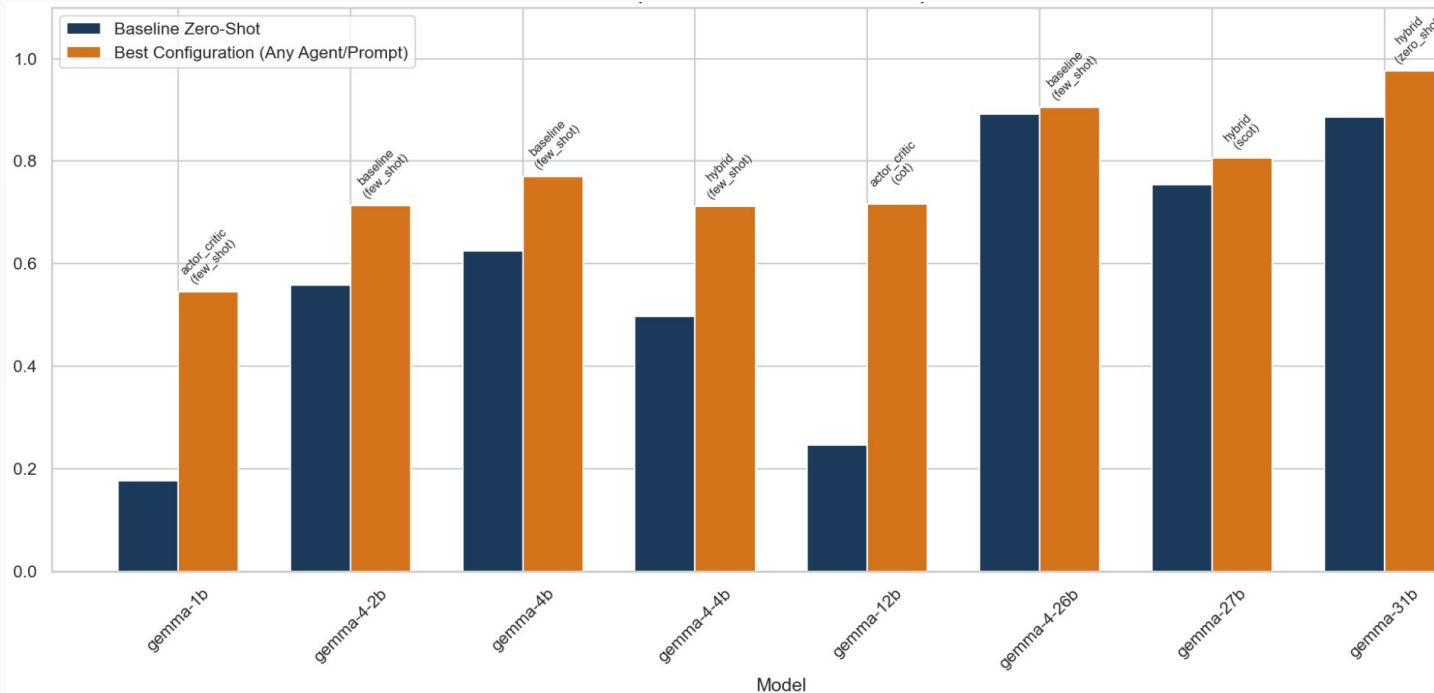
- **Line Coverage:** Fraction of executable lines hit.
- **Branch Coverage:** Fraction of control-flow branches traversed.
- **Line $\approx$ Branch:** Indicates linear code with few conditionals.
- **Line $\gg$ Branch:** Heavy conditionals with many missed edges.
- **Diversity Score:** Measures input/structural test variety.

Key Observations & Correlations

- **Logic Gap:** Coverage $\neq$ correctness; high coverage on buggy code is possible.
- **Coverage vs. PassRate (≈ 0.72)**
- **Coverage vs. Diversity (≈ 0.65)**
- **Coverage vs. Mutation (≈ 0.31)**



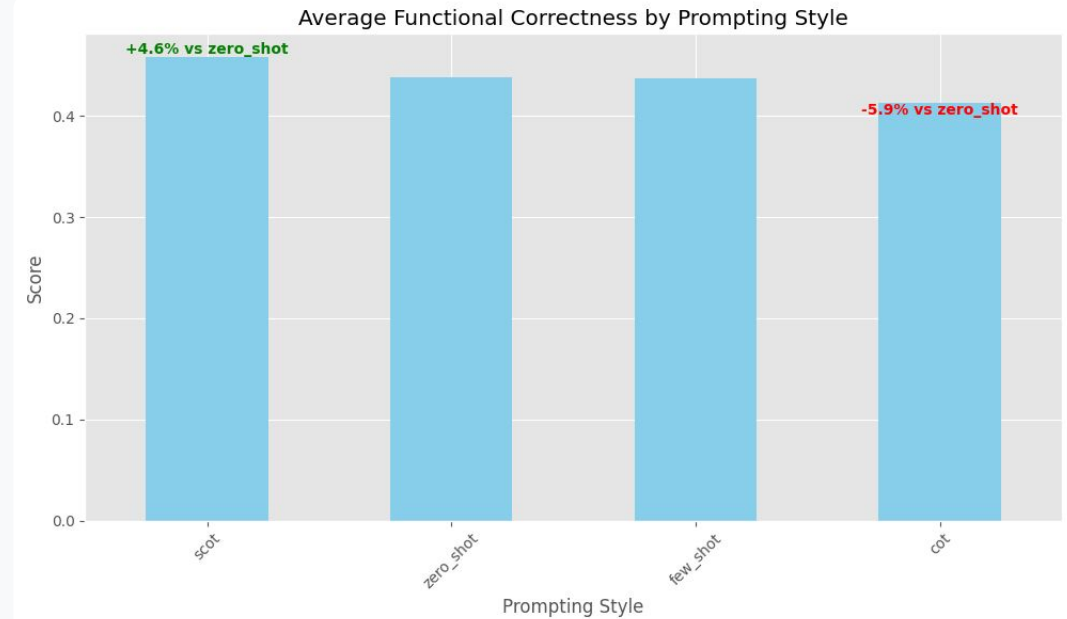
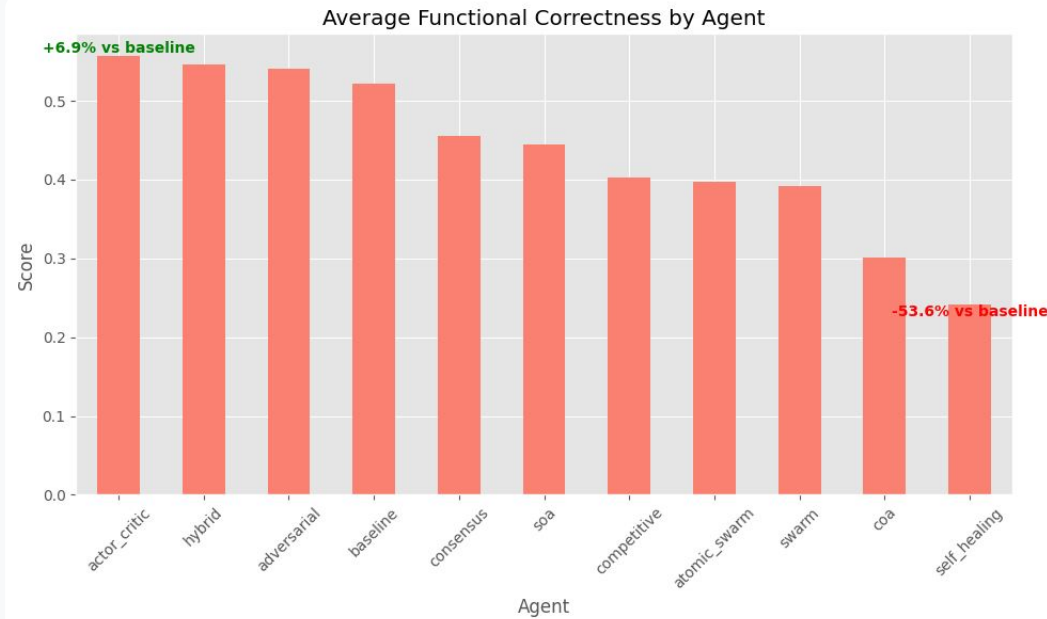
Takeaway: Coverage is necessary but not sufficient. It measures breadth, while mutation measures depth. Atomic Swarm can hit $\sim 100\%$ on buggy code via circular tests.



Key Insights

- Significant performance gains observed across all model sizes when switching from Baseline to Best Configuration.
- Smaller models benefit proportionally more from agent tuning, effectively narrowing the gap with larger models.
- Consistent outperformance of Zero-Shot configurations validates the impact of specialized agent architectures.

Takeaway: Optimized agent architectures can bridge the generational gap, occasionally allowing smaller or older models to match the baseline performance of larger, next-generation counterparts.



- **Spread is wider on agent axis than on prompt axis:** agent design has a greater impact than prompting alone, particularly regarding performance degradation.
- **SCoT leads on average:** While SCoT performs best, CoT shows significant degradation as it is optimized for general problem-solving rather than specific coding tasks.

Note: Results represent averages across all models; individual performance varies significantly by model.

Architecture Dominates the Prompt

Greater influence on results, but with higher structural risks

Results & Evidences

- Agent design can bridge model scale
- Logical Gap: high Coverage $\neq$ Pass Rate $\neq$ Mutation score

Trade-offs & Risks

- Specialized architectures can fail on generic tasks (e.g., Swarm for text generation)

No Free Lunch Theorem: No configuration is universal

Excellent architectures in specific contexts (e.g., Swarm for long-context) may be sub-optimal in others. There is no "silver bullet" for LLM-assisted software engineering; the choice strictly depends on the nature of the task and cost/performance constraints.

The Role of the Parser

The parser analyzes code generated by Large Language Models to extract logical structure and verify formal correctness before test execution.

Why Regex-Based Extraction?

- **Markdown Sanitization:** It targets and extracts code within triple backtick blocks (``python``), handling cases where the language tag is missing through non-greedy matching.
- **Robust Fallback logic:** If no Markdown blocks are detected, the parser treats the entire output as raw code, applying line-by-line cleaning to ensure structural integrity.

```
# Robust extraction and cleaning
import re

def extract_code(text: str) -> str:
    """Extract code from LLM Markdown blocks."""
    match = re.search(r"```(?:python)?\n?(.*?)```", text, re.DOTALL)
    if match: code = match.group(1).strip()
    else: code = text.strip()
    lines = code.splitlines()
    filtered = [l for l in lines if not l.lower().startswith("here is")]
    return "\n".join(filtered).strip()

def clean_test_code(code: str) -> str:
    """Removes placeholder imports from sandbox code."""
    pat = r"from\s+(your_module|test_module)\s+import"
    return re.sub(pat, "", code)
```

Security & Efficiency: By parsing the code first, we avoid executing broken scripts in our testing environment.

Key Takeaways

- **Agentic Architecture:** Greater potential for improvement, but also higher risk of degradation compared to standard prompting.
- **Model Size:** Remains an extremely critical factor in overall performance.
- **Deterministic Phase:** A Python-based parser is essential for reliable results.

Project Scope & Reliability

- **Limitations:** 25 tasks, 5 model sizes, within the HumanEval scope.
- **Reproducibility:** All configs, seeds, and prompts are versioned in Git.
- **Acknowledgment:** Special thanks to the supervisor and the entire project team.

"Agent design has a greater impact than prompting alone, but requires robust validation."



2026: Status Quo

In 2026, we used models up to 31B parameters.

Experimental efficiency was high, with total API costs kept under 4 euros.

2027: Projections

Expectation for slightly larger but significantly more efficient model families.

Anticipated improvements in agentic task benchmarks.

Vision: RLM

Application of Recursive Language Models (RLM) to tackle the Long Context barrier.

Overcoming the primary bottleneck of this project.

Key Objective: Solving the "Long Context" Challenge

Enabling more complex software engineering workflows through RLM architectures.



POLITECNICO DI TORINO

Large Language Models for Software Engineering · Academic Year 2025 / 2026

Thank you for your attention